



Algorithmen und Datenstrukturen

Leon Johann Brettin



Algorithmen und Datenstrukturen

8. Komplexität



Qualitätsmerkmale

- **Korrektheit**
 - Eigentlich immer nötig! Problem: Spezifikation?
- **Genauigkeit**
 - Bei numerischen Algorithmen, Videokompression,..
- **Einfachheit**
 - Leichter zu programmieren, weniger Fehler
- **Verständlichkeit**
 - Besser zu warten
- **Effizienz**
 - Speicherplatz
 - Laufzeit
 - Erst Korrektheit, dann Effizienz!



Optimierung

- Donald Knuth:
 - We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil.
 - Read [http://en.wikipedia.org/wiki/Optimization_\(computer_science\)](http://en.wikipedia.org/wiki/Optimization_(computer_science))
- Optimierung
 - Algorithmus
 - Implementierung / Quellcode
 - Compilieren
 - Assembler
 - Lesbarkeit und Wartbarkeit

Knuth, Donald.
Structured Programming with go to Statements,
ACM Journal Computing Surveys,
Vol 6, No. 4, Dec. 1974. p.268.



Laufzeiteffizienz

- Praktische Messung (Benchmark)
 - Abhängigkeit
 - Hardware
 - Programmiersprache, Compiler
 - Eingabe (Länge, spezielle Eingaben)
- Theoretische Analyse des Algorithmus
 - Unabhängig von Hardware, Programmiersprache.
 - Aber abhängig von Eingabe

⇒ Theoretische Analyse

- Anzahl der „elementaren“ Rechenoperationen
- Anzahl der Schleifendurchläufe
- Abhängig von der (Länge der) Eingabe



Schlimmster & durchschnittlicher Fall

- $T(n)$: Anzahl der Operationen, die bei einer Eingabe der Länge n maximal nötig sind (unabhängig von der genauen Eingabe).
Schlimmster Fall, **worst case**
- Alternative: Durchschnittlicher Fall (**average case**): Durchschnittliche Anzahl der Operationen bei Eingaben der Länge n .



Aufgabe

- Es sei ein eindimensionales Array $F []$ mit n Elementen (integer) gegeben.
- Entwerfen Sie einen Algorithmus, der feststellt, ob zwei (oder mehr) Arrayelemente übereinstimmen (nicht welche).
- Wie viele Vergleichsoperationen sind im „worst case“ erforderlich?



Suchen im unsortierten Array

- Gegeben Array mit n Elementen, suche Wert a.
- Anzahl der Vergleiche im
 - Schlimmsten, besten, durchschnittlichen Fall
 - Schlimmster: der letzte Vergleich ist nötig $V(n) = n$
 - Bester: der erste Vergleich reicht $V(n) = 1$
 - Durchschnittlicher: hängt von der Wahrscheinlichkeitsverteilung ab:
 - $V(n) = \sum_{i=1}^n i * P(arr[i] = a \wedge arr[j] \neq a \forall j = 1..i-1)$
 $+ n * P(arr[j] \neq a \forall j = 1..n)$
 - $P(arr[i] = a) = \frac{1}{n} \Rightarrow V(n) = \frac{1}{2}(n+1)$



Suchen im sortierten Array

- Gegeben Array mit n Elementen, suche Wert a .
- Anzahl der Vergleiche im
 - Schlimmsten Fall: $V(n) = \lfloor \log_2 n \rfloor + 1$
 - Besten Fall: $V(n) = 1$
 - Durchschnittlicher: hängt nicht von der Wahrscheinlichkeitsverteilung ab.
 - $V(n) \approx \lfloor \log_2 n \rfloor$



Asymptotische Analyse

- Genaue Laufzeit hängt ab von
 - Programmiersprache
 - Compiler (+ Compilereinstellungen)
 - CPU-Typ und -Geschwindigkeit
 - Speicher
 - sonstige Hardwaredetails
 - Größe der Eingabe
 - Details der Eingabe
- Abstraktion:
 - Laufzeit als Funktion der Größe der Eingabe
 - konstante Faktoren ignorieren.



Asymptotische Analyse

- Analyse der Komplexität: Angabe einer Funktion

$$f : \mathbb{N} \rightarrow \mathbb{N}$$

- als Maß für Aufwand
 - Bedeutung: $f(n) = a$
 - bei Problem der Größe n
 - beträgt der Aufwand a
- Problemgröße: Umfang der Eingabe, wie z.B. Anzahl der zu sortierenden oder zu durchsuchenden Elemente
- Aufwand: Rechenzeit (Abschätzung als Anzahl der Operationen wie z.B. Vergleiche), Speicherplatz



Schleifen

- Wie oft wird die Wertzuweisung „ $x := x + 1$ “ in folgenden Anweisungen ausgeführt?

```
x := x + 1
```

```
for i := 1 to n do x := x + 1 od
```

```
for i := 1 to n do  
  for j := 1 to n do  
    x := x + 1  
  od  
od
```



Schleifen

- Wie oft wird die Wertzuweisung „ $x := x + 1$ “ in folgenden Anweisungen ausgeführt?

$x := x + 1$

1 mal

for $i := 1$ to n do $x := x + 1$ od

n mal

```
for  $i := 1$  to  $n$  do
  for  $j := 1$  to  $n$  do
     $x := x + 1$ 
  od
od
```

n^2 mal



Schleifen

- Wie oft wird die Wertzuweisung „ $x := x + 1$ “ in folgenden Anweisungen ausgeführt?

```
doit()
```

```
loop()
```

```
for i := 1 to n do  
    loop()  
od
```

```
subroutine doit()  
begin  
    x := x + 1  
end
```

```
subroutine loop()  
begin  
    for i := 1 to n do  
        x := x + 1  
    od  
end
```



Schleifen

- Wie oft wird die Wertzuweisung „ $x := x + 1$ “ in folgenden Anweisungen ausgeführt?

`doit()` 1 mal

`loop()` n mal

`for i := 1 to n do`
 `loop()`
`od` n^2 mal

```
subroutine doit()  
begin  
  x := x + 1  
end
```

```
subroutine loop()  
begin  
  for i := 1 to n do  
    x := x + 1  
  od  
end
```



Aufwandsfunktion

- Aufwandsfunktion $f : \mathbb{N} \rightarrow \mathbb{N}$ meist nicht exakt bestimmbar, daher
 - Abschätzung des Aufwandes im schlechtesten Fall
 - Abschätzung des Aufwandes im Mittel
- und „ungefähres Rechnen in Größenordnungen“



Länge der Eingabe, Zahl der Operationen

- Zahl: Größe oder Länge der Darstellung
 - Unäre Darstellung oder Dualdarstellung?
- Operationen: Was ist elementar?
 - Addition zweier Zahlen? Multiplikation? Zuweisung?
 - Ein Schritt einer Turingmaschine
 - Increment/Decrement einer Zahl
- Theoretisch: logarithmisches Kostenmodell
 - Addition kostet so viel wie Anzahl der verwendeten Stellen.
- Praktisch:
 - Zahlenbereiche endlich (32 oder 64 Bit)
 - alle Operationen elementar



O-Notation

- Angabe der asymptotischen oberen Schranke für Aufwandsfunktion
 - Wachstumsgeschwindigkeit bzw. Größenordnung
- Asymptote: Gerade, der sich eine Kurve bei immer größer werdender Entfernung vom Koordinatenursprung unbegrenzt nähert
- einfache Vergleichsfunktion $g : \mathbb{N} \rightarrow \mathbb{N}$ für Aufwandsfunktion f mit
$$f(n) = O(g(n))$$



O-Notation

- Definition

- Seien $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ zwei Funktionen. Dann ist

$$f \in O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \in \mathbb{N}: \\ n > n_0 \Rightarrow f(n) \leq c \cdot g(n)$$

- Man sagt: f hat eine Komplexität von $O(g)$.
 - f wächst höchstens so schnell wie g .
 - Statt $f \in O(g)$ oft auch $f=O(g)$ geschrieben.
 - Landau Symbole $O, o, \Omega, \omega, \Theta$



O-Notation

$$f \in O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \in \mathbb{N}: \\ n > n_0 \Rightarrow f(n) \leq c \cdot g(n)$$

$$f \in O(g)$$

f wächst höchstens so schnell wie g



O-Notation

$$f \in O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \in \mathbb{N}: \\ n > n_0 \Rightarrow f(n) \leq c \cdot g(n)$$

$$\exists n_0 \in \mathbb{N}$$

Startpunkt n_0

Wir schauen auf große n , was vorher passiert ist egal.



O-Notation

$$f \in O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \in \mathbb{N}: \\ n > n_0 \Rightarrow f(n) \leq c \cdot g(n)$$

$$\exists c \in \mathbb{R}^+$$

Positive Konstante mit der wir g multiplizieren dürfen.

Wir dürfen g vergrößern, um f zu überdecken.



O-Notation

$$f \in O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \in \mathbb{N}: \\ n > n_0 \Rightarrow f(n) \leq c \cdot g(n)$$

$$\forall n \in \mathbb{N}$$

Für alle n Element der natürlichen Zahlen gilt



O-Notation

$$f \in O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \in \mathbb{N}: \\ n > n_0 \Rightarrow f(n) \leq c \cdot g(n)$$

$$n > n_0$$

Ab diesem Punkt gilt die Ungleichung



O-Notation

$$f \in O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \in \mathbb{N}: \\ n > n_0 \Rightarrow f(n) \leq c \cdot g(n)$$

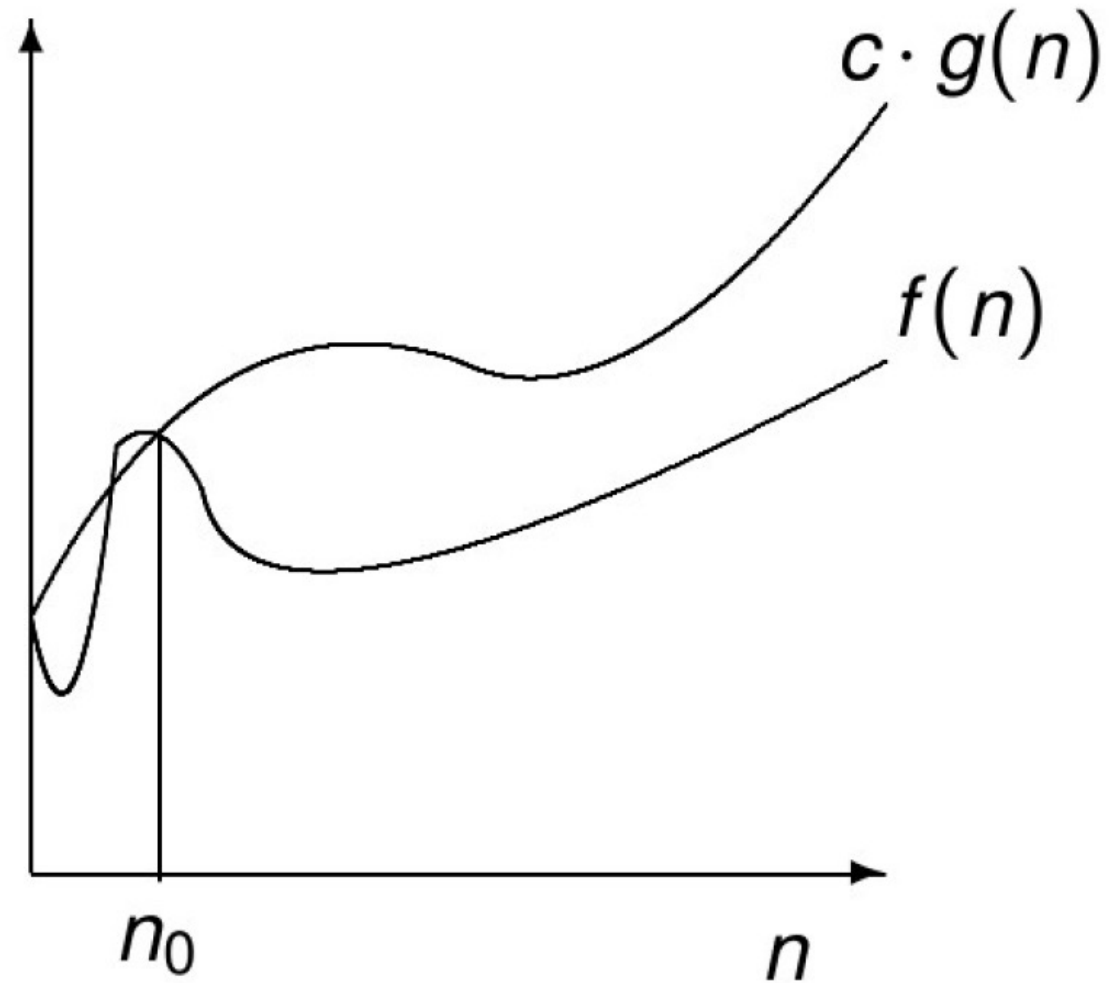
$$f(n) \leq c \cdot g(n)$$

Die Funktion f liegt unter einer skalierten Version von g



O-Notation

- $f \in O(g)$





Beispiele O-Notation

- $n+7 \in O(n)$
- $2^{20} n + 2^{30} \in O(n)$
- $f+g \in O(f)$ falls $g \in O(f)$
- $f \cdot g \notin O(f)$ falls $g \notin O(1)$



Beispiele O-Notation

- $n+7 \in O(n)$
 - Wähle $n_0 = 7$ und $c=2$, dann ist für $n>7$: $n+7 \leq n+n=2*n$
- $2^{20}n + 2^{30} \in O(n)$
 - Wähle $n_0 = 2^{10}$ und $c=2^{21}$, dann ist für $n>n_0$:
 - $2^{20}n + 2^{30} = 2^{20}(n + 2^{10}) \leq 2^{20}(n + n) = c*n$
- $f+g \in O(f)$ falls $g \in O(f)$
 - Aus $g \in O(f)$ folgt: es gibt n_{0g} und c_g . Wähle nun $n_0=n_{0g}$ und $c = c_g+1$.
Dann gilt $f+g \leq f+c_g*f = c*f$
- $f*g \notin O(f)$ falls $g \notin O(1)$.
 - Angenommen, $f*g \in O(f)$, dann gibt es n_0 und c mit
 - $f*g \leq c*f$. Damit gilt $g \leq c$ für alle $n>n_0$. Also $g \in O(1)$.



- $n+7 \in O(n)$



- $2^{20} n + 2^{30} \in O(n)$



- $f+g \in O(f)$ falls $g \in O(f)$



- $f+g \in O(f)$ falls $g \in O(f)$
- Was wir zeigen wollen
- $\exists n_0, c: f(n) + g(n) \leq c \cdot f(n)$
- Für alle $n < n_0$
- Die Summe darf höchstens ein konstantes Vielfaches von f sein.



- $f+g \in O(f)$ falls $g \in O(f)$
- $\exists n_0, c: f(n) + g(n) \leq c \cdot f(n)$

- $\exists, c_g, n_{0g}: g(n) \leq c_g f(n)$

- Für alle $n \geq n_0$

- g ist höchstens ein konstanter Faktor von f

Korrektur! Hier war vorher ein
'<' -Zeichen



- $f+g \in O(f)$ falls $g \in O(f)$
- $f + g \leq f + c_g f$
- Einsetzen der oberen Schranke für g
- f ausklammern
- $f + c_g f = (c_g + 1)f$
- Damit gilt
- $f + g \leq f + c_g f \leq (c_g + 1)f$



- $f+g \in O(f)$ falls $g \in O(f)$
- $f + g \leq (c_g + 1)f$
- Wahl der Konstanten
- $c = c_g + 1$
- $n_0 = n_{0g}$
- Dann gilt für alle $n > n_0$
- $f + g \leq cf$



- $f^*g \notin O(f)$ falls $g \notin O(1)$
- Angenommen
- $f \cdot g \in O(f)$
- Dann folgt:
- $\exists c > 0, n_0$:
- $f(n)g(n) \leq cf(n) \quad \forall n > n_0$
- $g(n) \leq c \quad \forall n > n_0$
- g ist durch eine Konstante beschränkt
- $g \in O(1)$



Beispiele O-Notation

- $n+7 \in O(n)$
 - Wähle $n_0 = 7$ und $c=2$, dann ist für $n>7$: $n+7 \leq n+n=2*n$
- $2^{20}n + 2^{30} \in O(n)$
 - Wähle $n_0 = 2^{10}$ und $c=2^{21}$, dann ist für $n>n_0$:
 - $2^{20}n + 2^{30} = 2^{20}(n + 2^{10}) \leq 2^{20}(n + n) = c*n$
- $f+g \in O(f)$ falls $g \in O(f)$
 - Aus $g \in O(f)$ folgt: es gibt n_{0g} und c_g . Wähle nun $n_0=n_{0g}$ und $c = c_g+1$.
Dann gilt $f+g \leq f+c_g*f = c*f$
- $f*g \notin O(f)$ falls $g \notin O(1)$.
 - Angenommen, $f*g \in O(f)$, dann gibt es n_0 und c mit
 - $f*g \leq c*f$. Damit gilt $g \leq c$ für alle $n>n_0$. Also $g \in O(1)$.



O-Notation praktisch

- Weglassen von multiplikativen Konstanten
 - $2 \cdot n \in O(n)$
- Weglassen von additiven Konstanten (Startup-Kosten)
 - $n + c \in O(n)$
- Beschränkung auf höchsten Exponenten
 - $n^2 + n \in O(n^2)$
- Basis des Logarithmus ist unerheblich
 - $\log_2 n \in O(\log n)$
 - $\log_b n = \log_a n \cdot \log_b a$



Typische O-Notation

- $O(f)$ ist nur Obergrenze
 - f sollte einfach sein
 - Keine konstanten Faktoren
 - Keine langsam wachsenden Summanden
 - Möglichst dicht!
 - Teilweise sehr langsam wachsende Faktoren ignoriert:
 - $O(n^2 \log(\log n))$
- $O(1)$
 - $O(\log n)$
 - $O(\sqrt{n})$
 - $O(n)$
 - $O(n \log n)$
 - $O(n^2)$
 - $O(n^k)$
 - $O(2^n)$
 - ...



Typische Zeitkomplexitäten

- **$O(1)$ Konstante Komplexität:** Einige Suchverfahren für Tabellen (Hashing)
- **$O(\log n)$ Logarithmische Komplexität:** Sehr günstig, da die Laufzeit erheblich schwächer wächst als n . Binäre Suche
- **$O(n)$ Lineare Komplexität:** Immer noch günstig. Auf jedes der n Eingabeelemente entfällt eine feste Verarbeitungszeit.
- **$O(n \log n)$ Leicht überlineare Komplexität:** Tritt oft auf, wenn ein Problem fortgesetzt in Teilprobleme zerlegt wird, die unabhängig voneinander gelöst werden. Beispiel: Sortieren
- **Quadratische Komplexität:** Ungünstig. Zwei geschachtelte Schleifen, die auf alle n Daten angewandt werden.
- **Kubische Komplexität:** Sehr ungünstig und nur auf kleine n anwendbar.
- **Exponentielle Komplexität:** Katastrophal für große n . Tritt auf, wenn zur Lösung des Problems alle Möglichkeiten exhaustiv geprüft werden müssen. Viele Optimierungsprobleme!



Obere, untere und scharfe Schranke

- Obere Schranke
- $f \in \mathcal{O}(g) \Leftrightarrow \exists n_0 \in \mathbb{N} \exists c \in \mathbb{R}^+ \forall n \in \mathbb{N}: n > n_0 \Rightarrow f(n) \leq c * g(n)$
- Untere Schranke
- $f \in \mathcal{\Omega}(g) \Leftrightarrow \exists n_0 \in \mathbb{N} \exists c \in \mathbb{R}^+ \forall n \in \mathbb{N}: n > n_0 \Rightarrow f(n) \geq c * g(n)$
- Asymptotisch scharfe Schranke
- $f \in \Theta(g) \Leftrightarrow f \in \mathcal{O}(g) \wedge f \in \mathcal{\Omega}(g)$
 $\Leftrightarrow \exists n_0 \in \mathbb{N} \exists c_u \in \mathbb{R}^+ \exists c_o \in \mathbb{R}^+ \forall n \in \mathbb{N}: n > n_0 \Rightarrow c_u * g(n) \leq f(n) \leq c_o * g(n)$



Laufzeiten verschiedener Algorithmen

- Annahme: Ein Schritt $\triangleq$ 1 Mikrosekunde

N=	20	50	100	200	500	1000	10000
$1000 \cdot n$	0,02 s	0,05 s	0,10 s	0,20 s	0,50 s	1 s	10 s
$1000 \cdot n \log n$	0,09 s	0,28 s	0,66 s	1,53 s	4 s	10 s	2 min
$100 \cdot n^2$	0,04 s	0,25 s	1 s	4 s	25 s	1 min	2 h
$10 \cdot n^3$	0,08 s	1,25 s	10 s	1 min	20 min	2 h	17 Wochen
2^n	1,05 s	35 J	$4 \cdot 10^{17}$ J				

- Schnellerer Rechner! Gute Idee, bringt aber nur konstanten Faktor



Lösbare Problemgröße

- Algorithmus benötigt $t(n)$ Schritte
- Maschine leistet f Schritte/sek
- Laufzeit $T(n) = \frac{t(n)}{f}$
- Welche Problemgröße ist lösbar (in k Sekunden)?

- $k=T(n)$

- $n=t^{-1}(kf)$

	k=	1 s	10 s	60 s	1 h	1 t	1 J
$1000*n$		1000	10000	60000	$3.6*10^6$	$8.6*10^7$	$3.1*10^{10}$
$1000*n \log n$		140	1003	4895	$2*10^5$	$4*10^6$	$1*10^9$
$100*n^2$		100	316	774	6000	$3*10^4$	$5.6*10^5$
$10*n^3$		46	100	181	711	2052	14664
2^n		20	23	25	31	36	44



Laufzeitanalyse & Komplexitätstheorie

- Laufzeit von **Algorithmen**
 - Gegeben ein Algorithmus, wie lange läuft er für eine Eingabe (ein Problem) der Größe n ?
 - Unabhängig von der Maschine, der Sprache, etc..
 - Konstante Faktoren werden ignoriert.
 - $O(n)$ = Menge aller Funktionen, die asymptotisch nicht schneller wachsen als n .
 - $O(n) = O(2n)$
 - $O(n+1) = O(n)$
 - Durchschnittlich oder schlimmstenfalls
- Komplexität von **Problemen**
 - Laufzeit des besten denkbaren Algorithmus
 - ...und wenn der beste nicht bekannt ist (oder beweisbar nicht existiert)?



P vs. NP

- **P**: Die Klasse aller Probleme, die mit einer deterministischen Turingmaschine in Zeit $O(n^p)$ gelöst werden können.
- **NP**: Die Klasse aller Probleme, die mit einer nichtdeterministischen Turingmaschine in Zeit $O(n^p)$ gelöst werden können.
- **$P \subseteq NP$** per Definition.
- Ein Problem in NP kann deterministisch in **exponentieller Zeit** gelöst werden.
 - z.B. rekursiv den Baum der nichtdeterministischen Entscheidungen absuchen.
 - Leider nicht realistisch für größere Probleme!
- **$P \neq NP$** ? Ungeklärt! vermutlich ja, aber...



Erfüllbarkeitsproblem SAT

- Gegeben: aussagenlogischer Ausdruck mit
 - Variablen a, b, c etc.,
 - logischen Operatoren $\wedge, \vee, \neg$ und Klammern
- Gesucht: Algorithmus, der prüft, ob Formel erfüllbar ist, d.h., ob Belegung der Variablen mit booleschen Werten `true` und `false` existiert, so dass Formel `true` liefert
- Lösungsidee: Testen aller Belegungen
 - Aufwand?



P vs. NP: wichtige Konzepte

- **Reduktion von Problemen**
 - Lösung von Problem 1 kann auf die Lösung von Problem 2 zurückgeführt werden.
 - Falls Reduktion nur polynomiellen Aufwand erfordert.
- **NP-Vollständigkeit**
 - NP-vollständige Probleme sind so mächtig (schwierig), dass jedes Problem in NP auf diese reduziert werden kann.
- Wenn **ein** NP-vollständiges Problem $\in P$ ist, dann sind **alle** $\in P$, und $P = NP$.
- NP-vollständige Probleme vermutlich nicht in $O(n^p)$ lösbar.
 - Travelling Salesman Problem
 - SAT (Satisfiability of logical formulas)
 - >1000 andere



Pragmatische Analyse von Algorithmen

- Bestimmung der Laufzeitkomplexität von Algorithmen
 - gleich mit der O-Notation!
- for-Schleifen
 - Laufzeit: maximale Laufzeit der inneren Anweisung * Anzahl der Iterationen

```
for i := 1 to n do  
    a[i] := 0;  
od
```

- wenn innere Operation $O(1)$, dann $n \cdot O(1) = O(n)$



Geschachtelte for-Schleifen

- Aufwand Laufzeit := Laufzeit der inneren Anweisung * Produkt der Größen aller Schleifen!

```
for i := 1 to n do
  for j := 1 to n do
    k := k + 1
  od
od
```

- $n * n * O(1) = O(n^2)$



Nacheinanderausführung

- Beispiel

```
for i := 1 to n do
    a[i] := 0;
od;
for i := 1 to n do
    for j := 1 to n do
        a[i] := a[i] + a[j] + i + j
    od
od
```

- $O(n) + O(n^2) = O(n^2)$



If-else Bedingungen

- Aufwand Laufzeit :=
Aufwand für Test + max (Aufwand für A_1 , Aufwand für A_2)
- Beispiel

```
if x > 100 then
  y := x
else
  for i := 1 to n do
    if a[i] > y then y := a[i] fi
  od
fi
```

- $O(n)$



Zusammenfassung

- Komplexität
 - O-Notation
 - Komplexitätsklassen
 - P- vs. NP-Probleme



Welche der folgenden Aussagen ist falsch?

- A. Die Laufzeiteffizienz eines Algorithmus kann unabhängig von der Hardware beurteilt werden.
- B. Die Laufzeit im schlimmsten Fall ist immer mindestens so groß wie die Laufzeit im durchschnittlichen Fall (bei gleicher Eingabegröße).
- C. Je größer die Eingabe, desto größer die Laufzeit eines Algorithmus.
- D. Der beste Fall ist meist nicht repräsentativ für die Komplexität eines Algorithmus



Markieren Sie alle Formeln $\in O(n)$

$$42\sqrt{n} + 0.1n$$

$$42n^2 + 0.5n$$

$$13n + \frac{15}{n} + \frac{17}{n^2}$$

$$\sum_{i=1}^n i$$

$$\sum_{i=1}^n \frac{3i}{n}$$

$$\sum_{i=1}^n i\sqrt{n}$$



In welcher Komplexitätsklasse liegt

$$\sqrt{k} + 0.3n + \sum_{i=1}^k i$$

- A. $O(\sqrt{n})$
- B. $O(\sqrt{k})$
- C. $O(k)$
- D. $O(k)+O(n)$
- E. $O(k^2)+O(n)$
- F. $O(k^2)+O(n)+O(\sqrt{k})$



In welcher Komplexitätsklasse liegt

$$\sqrt{x} + 0.3x + 15x^3 + \sum_{i=1}^x i$$

- A. $\Theta(\sqrt{x})$
- B. $\Theta(x)$
- C. $\Theta(x \sqrt{x})$
- D. $\Theta(x^2)$
- E. $\Theta(x^3)$



Beweis für $O(x^3)$ und $\Omega(x^3)$

$$\sqrt{x} + 0.3x + 15x^3 + \sum_{i=1}^x i$$